School of Informatics & Cybersecurity - Assessment Submission Coversheet

Student Information

Student Number:	B00176896
Student Name:	Miguel Caracuel Millan

Module Information

Module Title:	Fundamentals of AI and Data Science
Module Code:	COMPH2706

Assessment Information

Assessment Title:	Project

Declaration

✔ I confirm that I have read and understood the University's Academic Integrity Policy.

https://www.tudublin.ie/explore/about-the-university/academic-affairs/academic-quality-assurance-and-enhancement/academic-integrity/

✔ I have completed the TU Dublin AI Driving License theory test.

https://www.tudublin.ie/explore/about-the-university/academic-affairs/lta/lta-resources/artificial-intelligence-driving-license/

✔ I declare that this assessment submission is my own work.

✔ I confirm that I have read and understood the AI traffic light system used as part of this assessment brief and I have adhered to the level of AI usage allowed for this assessment.

Student signature:	Miguel Caracuel Millan
Date:	27/04/2026

I am working as a data scientist, and I am working for an organization that has been experiencing several cyber-attacks that have cost several millions of financial losses, my main objective is to analyze the network traffic patterns to detect attacks before they cause any damage.

The objective of this project is to analyze the dataset Dataset1_even_students.csv and identify attack patterns and develop a model to detect intruders efficiently.

I will be doing it using the Cross-Industry Standard Process for Data Mining framework.

Process:

First, dependencies and libraries are installed and runed:

```
import pandas as pd           # DataFrames and CSV loading
import numpy as np            # Numerical operations

# — Visualisation ————————————————————————————
import matplotlib.pyplot as plt   # Base plotting
import seaborn as sns             # Statistical visualisations (built on matplotlib)

# — Machine learning models ——————————————————————
from sklearn.linear_model import LinearRegression
from sklearn.neighbors import KNeighborsRegressor
from sklearn.tree import DecisionTreeRegressor
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder
from sklearn.metrics import accuracy_score
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import classification_report, accuracy_score
from sklearn.impute import SimpleImputer
from sklearn.model_selection import cross_val_score
# — Evaluation metrics ———————————————————————————
from sklearn.metrics import r2_score, mean_squared_error
```

After this we got a quick overview of the dataset by importing the dataset and using df.head() to see the database's 5 first columns.

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.preprocessing import StandardScaler, LabelEncoder
from sklearn.model_selection import train_test_split

df = pd.read_csv('Dataset1_even_students.csv')
df.head()
```

	session_id	protocol	device_type	vpn_status	region	login_attempts	failed_logins	bytes_sent	bytes_received	packet_rate	co
0	S000000	TCP	IoT	On	APAC	1	0	3688.587042	7658.783289	184.351121	
1	S000001	ICMP	Server	Off	EU	1	2	4200.249009	7473.548707	127.567880	
2	S000002	TCP	Server	Off	APAC	1	0	5681.503452	6941.575921	206.334163	
3	S000003	ICMP	Server	On	US	1	0	7234.103007	10003.414511	106.159532	
4	S000004	Udp	Workstation	Off	ME	1	0	5798.763455	7768.487288	116.326988	

As we can see we have:

- Session_id which has an identifier for each row.
- Protocol which informs us about the internet protocol used.
- Device_type: which says what type of device is each one.
- VPN_status: this column informs us if the VPN is on or off.
- Region: which informs us about the region where it is operating.
- Login_attempts
- Failed_logins
- Bytes_sent
- Bytes_received
- Packet_rate

```
id()
```

connection_count	tcp_flags	port_number	session_retries	signal_strength	privilege_level	anomaly_score	session_duration
2	SYN	143	1	-58.159386	guest	0.632684	60.122861
5	SYN-ACK	25	0	-61.394428	guest	0.665009	88.064962
3	SYN	3389	1	-60.409574	user	0.024022	91.460489
7	SYN-ACK	21	3	-49.082725	user	0.388327	92.349695
1	FIN	143	0	-56.260717	user	0.266618	91.778194

- Connection_count: where we can see how many connections are.
- TCP_flags
- Port_number
- Session_retries
- Signal_strength
- Privilege_level: which shows what level of access each user has

- Anomaly_score:
- Session_duration: it shows how much time the session lasted.

ion_count	tcp_flags	port_number	session_retries	signal_strength	privilege_level	anomaly_score	session_duration	is_attack
2	SYN	143	1	-58.159386	guest	0.632684	60.122861	Benign
5	SYN-ACK	25	0	-61.394428	guest	0.665009	88.064962	Attack
3	SYN	3389	1	-60.409574	user	0.024022	91.460489	Benign
7	SYN-ACK	21	3	-49.082725	user	0.388327	92.349695	Benign
1	FIN	143	0	-56.260717	user	0.266618	91.778194	Benign

- Is_attack: this column is one of the most important ones, because it says whether a connection is an attack or not.

```
print("Summary")
display(df.describe())

print("\nDataset information")
display(df.info())

print("\nClass distribution")
print(df['is_attack'].value_counts(normalize=True))
```

Summary

	login_attempts	failed_logins	bytes_sent	bytes_received	packet_rate	connection_count	port_number	session_retries	signal_strength	anomaly_score	session_duratic
count	6000.000000	6000.000000	6000.000000	5580.000000	6000.000000	6000.000000	6000.000000	6000.00000	5580.000000	6000.000000	6000.0000
mean	1.972500	1.020667	5275.753584	5981.881281	126.137459	4.963500	472.757000	1.01950	-59.996008	0.283358	92.9144
std	1.415537	1.004527	3132.622483	1994.673822	71.772746	2.268485	1033.630588	1.00081	9.981340	0.159846	16.5950
min	0.000000	0.000000	100.000000	100.000000	23.362556	0.000000	21.000000	0.00000	-94.085793	0.002210	32.7363
25%	1.000000	0.000000	4009.308961	4662.928544	99.854757	3.000000	25.000000	0.00000	-66.789129	0.157360	81.4282
50%	2.000000	1.000000	5013.713497	5992.395457	120.528816	5.000000	80.000000	1.00000	-60.054692	0.260576	92.8785
75%	3.000000	2.000000	6060.269321	7333.109710	140.300293	6.000000	143.000000	2.00000	-53.439260	0.389571	104.1940
max	9.000000	7.000000	48743.941649	13019.206996	1147.060032	15.000000	3389.000000	6.00000	-25.139054	0.824099	156.4313

After having a quick review of the dataset, we went more into it looking at whose dataset.

Here we can see different things starting with the differences in count row, here we can see that almost every column has a value of 6000 in count but there are 2 of them that have different values: bytes_received and signal_strength, this means that we have missing values in both columns.

```
Dataset information
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 6000 entries, 0 to 5999
Data columns (total 19 columns):
 #   Column            Non-Null Count  Dtype
---  ------            --------------  -----
 0   session_id        6000 non-null   object
 1   protocol          5580 non-null   object
 2   device_type       6000 non-null   object
 3   vpn_status        6000 non-null   object
 4   region            6000 non-null   object
 5   login_attempts    6000 non-null   int64
 6   failed_logins     6000 non-null   int64
 7   bytes_sent        6000 non-null   float64
 8   bytes_received    5580 non-null   float64
 9   packet_rate       6000 non-null   float64
 10  connection_count  6000 non-null   int64
 11  tcp_flags         6000 non-null   object
 12  port_number       6000 non-null   int64
 13  session_retries   6000 non-null   int64
 14  signal_strength   5580 non-null   float64
 15  privilege_level   6000 non-null   object
 16  anomaly_score     6000 non-null   float64
 17  session_duration  6000 non-null   float64
 18  is_attack         6000 non-null   object
dtypes: float64(6), int64(5), object(8)
memory usage: 890.8+ KB
None
```

In this image which shows us the number of columns that contain each type of data such as:

Numerical columns: we have 11 of them (int64 and float64)

Object columns: we have 8 of them (protocol, is_attack…)

It also shows us new information that was not shown in the image from before. It also says that we have missing values in the column “Protocol”.

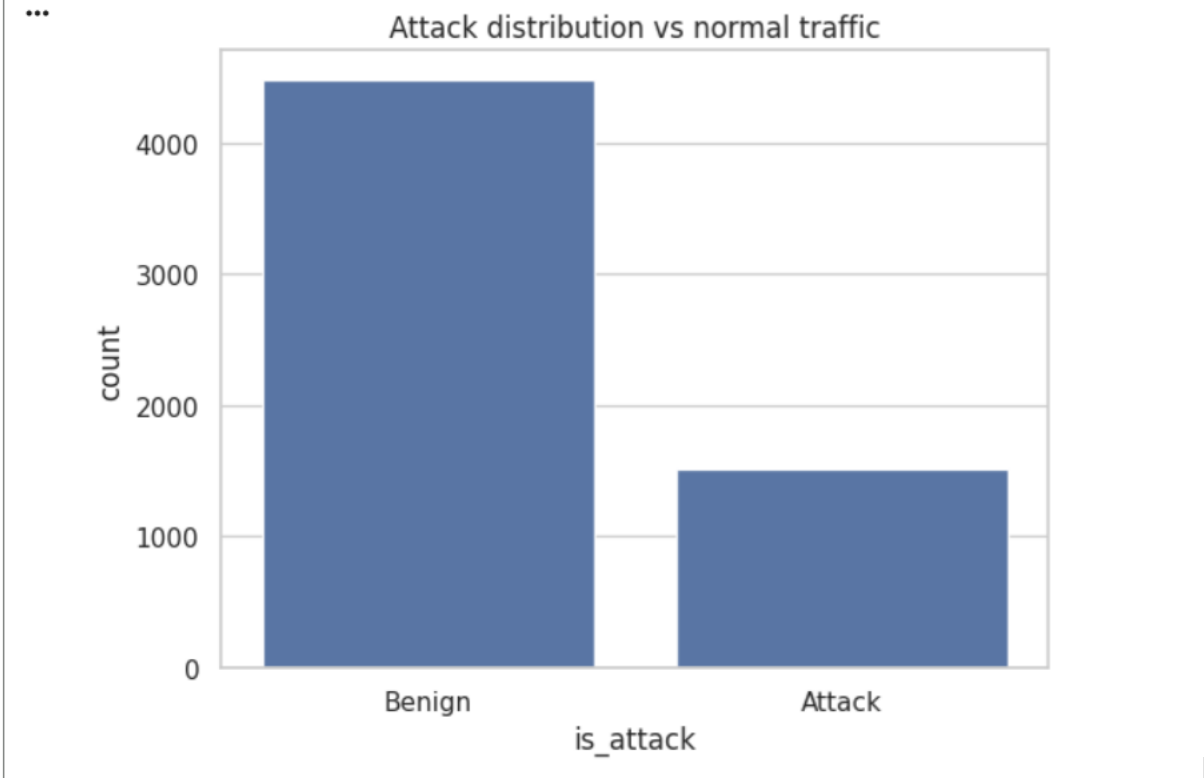
All the columns that have missing values surprisingly have the same number of missing values.

```
Class distribution
is_attack
Benign    0.746833
Attack    0.253167
Name: proportion, dtype: float64
```

We also wanted to know the percentage of the Bening vs Attack.

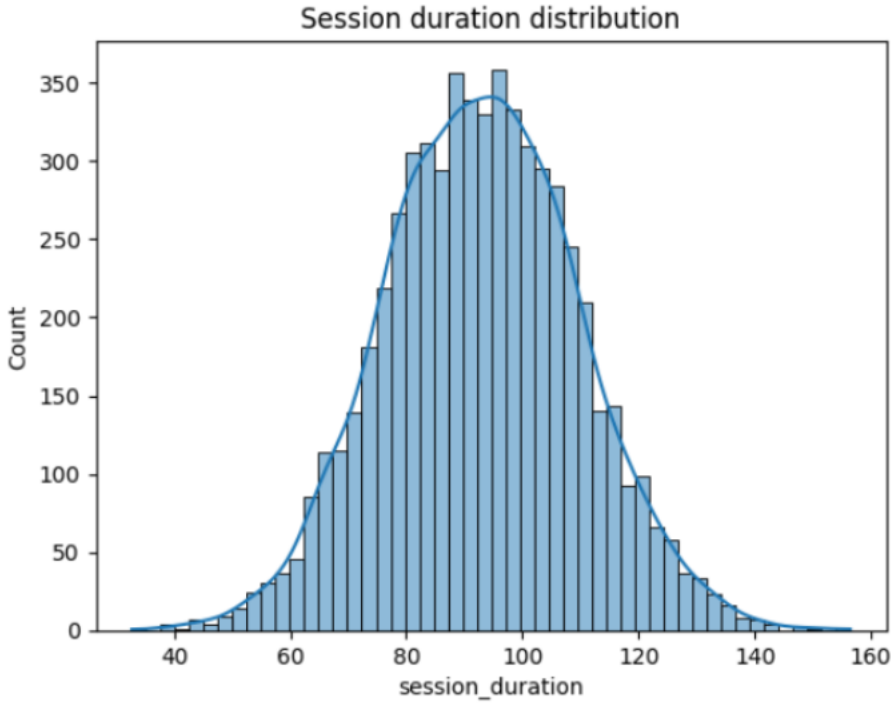
As we can see, the percentage is ~75%-25% being 75% the Bening and 25% the Attack value.

```
plt.show()
```



Here we can also see how the percentage of the attacks vs benign are in a more visual way.

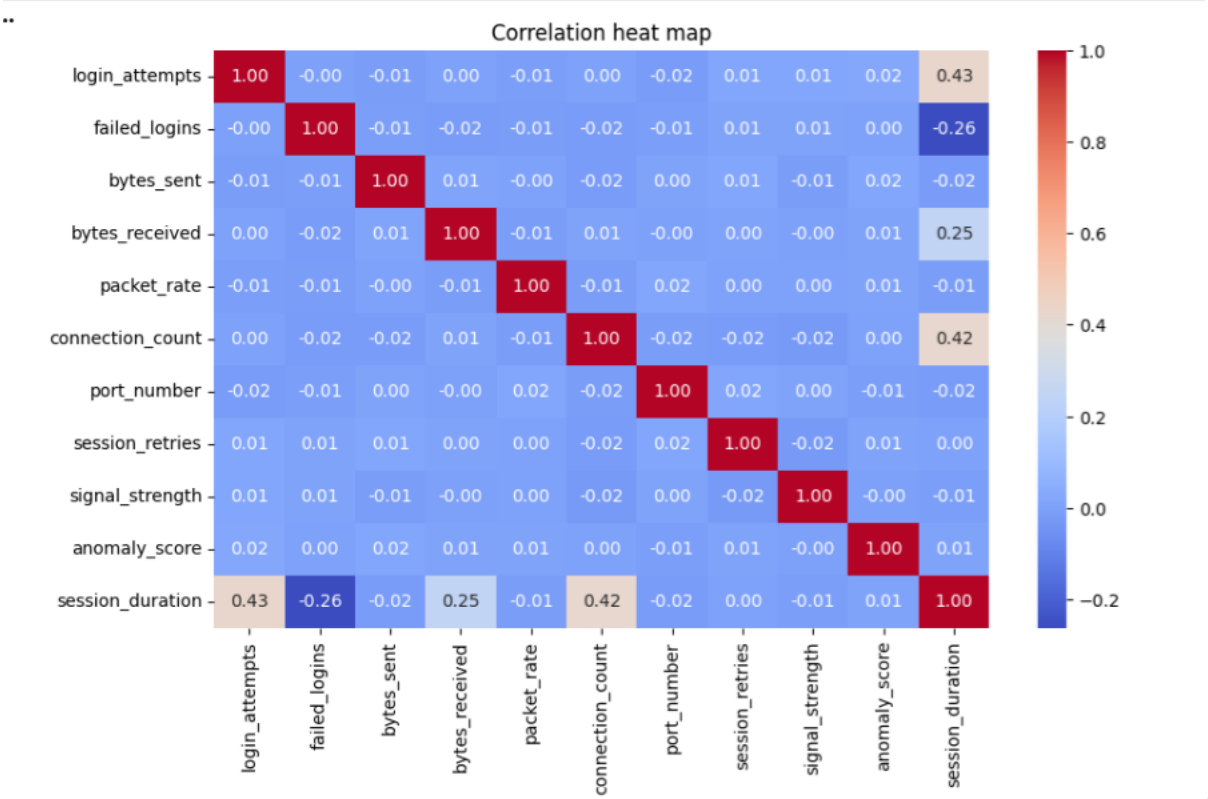
A 25% of attacks is an extremely high rate for a real financial network, the model should priorize Recall over anything else because a single attack passing the security measures could result in millions of financial looses



Now we can see a graph that shows us the distribution of session distribution, they are in a Gauss distribution (normal) not skewed which means that most of the data is concentrated in the middle.

We can see that most of the sessions take between 80 and 100-110 so if any session takes a number that is far from those numbers, for example 5 or 800 this would be considered an outlier, and attacks often take place in an outlier value.

This graph shows a normal session duration.



Now we elaborate on a correlation matrix which helps us to understand how the database is.

Almost all benign values are close to 0, which means that they are like the standard value while attack values are more variated and unpredictable.

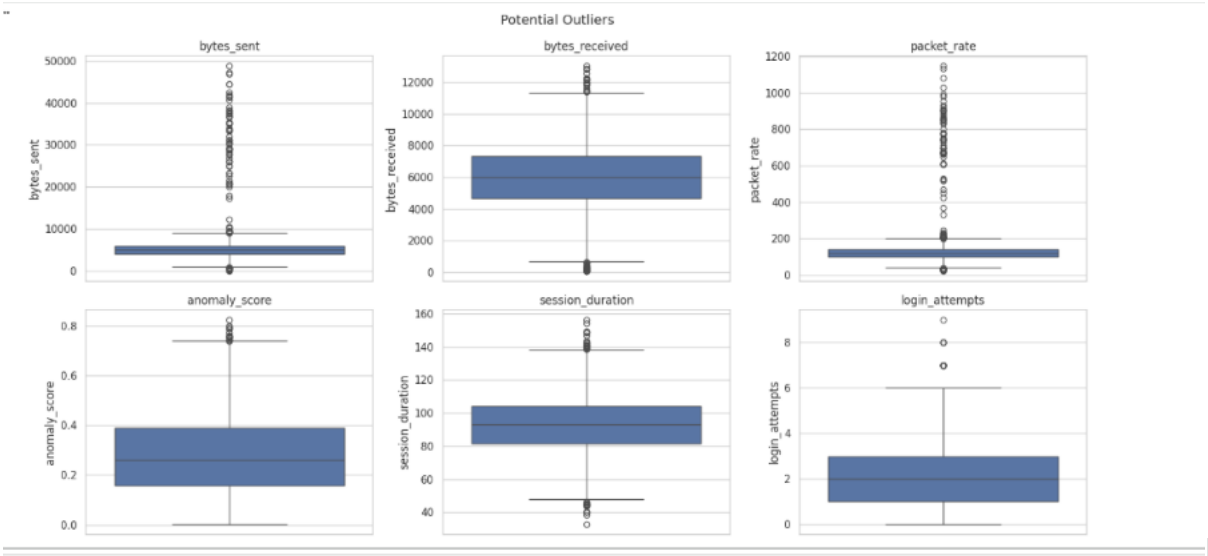
We can see that in the heatmap we have 0.42 and 0.43 for connection_count and for loggin_attempts, this means that attackers could be trying to log into the system multiple times as if it was a bot trying to accomplish a brute force attack

So, with this we can deduce that loggin_attempts and session_duration are the highest correlated (0.43) values, which means that with a greater number of attempts to loggin more time it takes the session.

Connection_count and session_duration(0.42), when there are more connections, the session is longer.

We also have a high correlation between connection_count and session_duration it means that when there are more connections the sessions last longer, which is normal.

And there also is a negative correlated correlation this being failed_logins and session_duration (-0.26). This means that when there are more failed login attempts, less time it takes the session. This could mean some kind of attack such as brute force.



Here we are using Boxplots as we can see there are outlier values in each variable.

The most important ones are bytes sent, which has a big amount of data sent, over 10000 bytes sent, which could mean that there is a data exfiltration attack.

Bytes received is another important one because it also has big outliers meaning that the system is downloading a big amount of content.

Packet_rate outliers show that it could be port scan due to its high outliers amount.

Anomaly_score outliers mean that it is possible that those outliers are due to suspicious events.

Session_duration's outliers mean that there are sessions longer and shorter than normal.

Login_attempts outliers could mean a brute force attack.

Data cleaning

```
null values per column:
session_id            0
protocol            420
device_type           0
vpn_status            0
region                0
login_attempts        0
failed_logins         0
bytes_sent            0
bytes_received      420
packet_rate           0
connection_count      0
tcp_flags             0
port_number           0
session_retries       0
signal_strength     420
privilege_level       0
anomaly_score         0
session_duration      0
is_attack             0
dtype: int64
duplicated rows: 0
```

As we can see in this image, first we count how many null values there are in the dataset, then we have null values in the columns protocol (420), bytes_received(420) and signal_strength(420), as we can see all of them have the exact same number of null values, this could be due to an error with the system.

Then we check the number of duplicated values that exist in the dataset.

We decided not to erase all this data because in an organizational environment these 420 logs are very important. When we put it as the mean, the AI system learns new patterns for cyber-attacks in other variables, minimizing the risk.

```
df_baseline = df.copy()
```

Eliminate Nulls

```
df_baseline.dropna(inplace=True)
```

o(), to check if the rows were really created and as we can see, there are:

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 6000 entries, 0 to 5999
Data columns (total 21 columns):
 #   Column              Non-Null Count  Dtype  
---  ------              --------------  -----  
 0   session_id          6000 non-null   object 
 1   protocol            6000 non-null   object 
 2   device_type         6000 non-null   object 
 3   vpn_status          6000 non-null   object 
 4   region              6000 non-null   object 
 5   login_attempts      6000 non-null   int64  
 6   failed_logins       6000 non-null   int64  
 7   bytes_sent          6000 non-null   float64
 8   bytes_received      6000 non-null   float64
 9   packet_rate         6000 non-null   float64
 10  connection_count    6000 non-null   int64  
 11  tcp_flags           6000 non-null   object 
 12  port_number         6000 non-null   int64  
 13  session_retries     6000 non-null   int64  
 14  signal_strength     6000 non-null   float64
 15  privilege_level     6000 non-null   object 
 16  anomaly_score       6000 non-null   float64
 17  session_duration    6000 non-null   float64
 18  is_attack           6000 non-null   object 
 19  failed_login_ratio  6000 non-null   float64
 20  network_intensity   6000 non-null   float64
dtypes: float64(8), int64(5), object(8)
memory usage: 984.5+ KB
```

Rows number 19 and 20 failed_login_ratio and network_intensity. And we also ensure that they are not null or missing values.

```python
X_base = df_baseline.drop(['is_attack', 'session_duration'], axis=1)
y_base = df_baseline['is_attack']

X_train_b, X_test_b, y_train_b, y_test_b = train_test_split(
    X_base, y_base, test_size=0.2, random_state=42, stratify=y_base
)

dt_baseline = DecisionTreeClassifier(max_depth=5, random_state=42)
dt_baseline.fit(X_train_b, y_train_b)
y_pred_base = dt_baseline.predict(X_test_b)

print("Baseline: Decision Tree")
print(classification_report(y_test_b, y_pred_base))
```

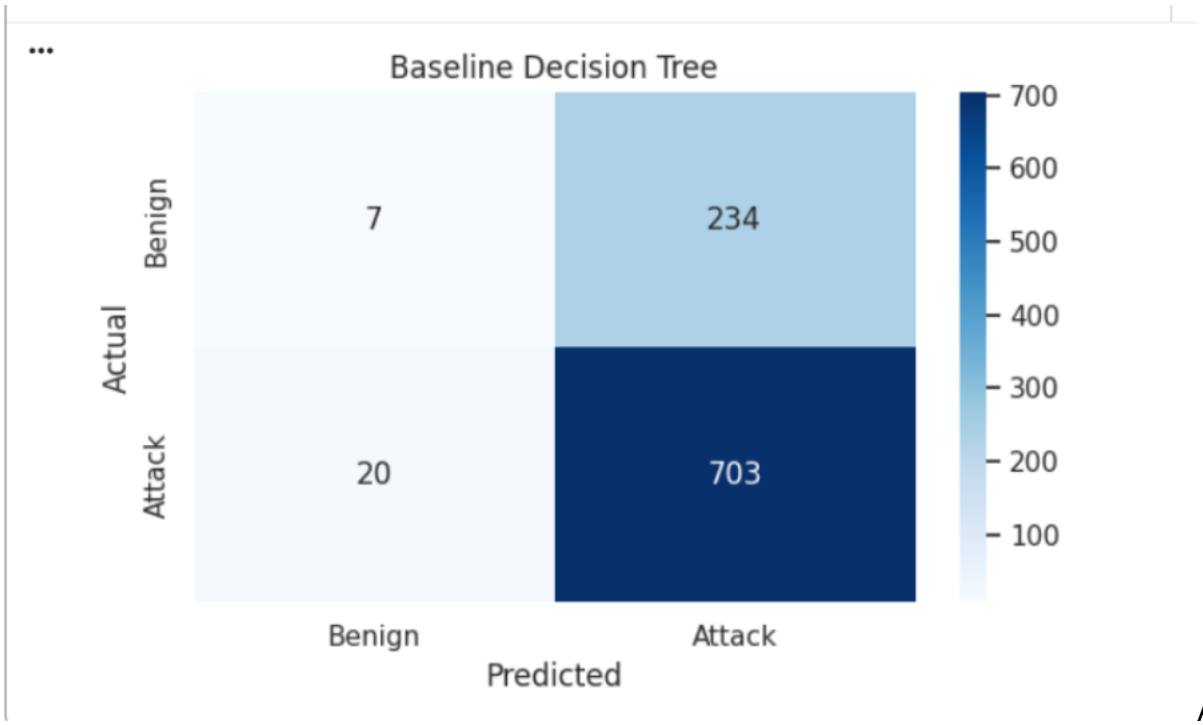
```
Baseline: Decision Tree
              precision    recall  f1-score   support

           0       0.26      0.03      0.05       241
           1       0.75      0.97      0.85       723

    accuracy                           0.74       964
   macro avg       0.50      0.50      0.45       964
weighted avg       0.63      0.74      0.65       964
```

Here we are trying to predict if is_attack is really an attack or benign traffic.

But we have a problem, data imbalance. There are more examples of class 1 than examples of class 0. The result of this decision tree makes us see that the model could be good because it has 74% accuracy it is not true, for class 0 recall is 0.03 which means that the model fails to detect 97% of the data given. The model learned to predict variable “1” most of the time because the dataset is giving a bigger amount of number 1 variables than number 0.



As we can see in this confusion matrix, the model detected 703 attacks correctly (this is good because we can see that the model detects attacks correctly) and the model only detected 7 benign that were really benign which is a very low compared to the 234 benign that it detected as an attack.

The conclusion in this part is that the model is not functioning correctly because it is almost always trying to read the benign data as an attack due to its laziness.



Improved model

Here we start with the improved model, first we create a copy of the database stored in df.copy, then we create new variables that are useful in the database context: failed_login_ratio which is the result of failed_login/login_attempts and network_intensity which is the result of bytes_sent + bytes_received /

session_duration and then we are filling divisions /0 with 0 as result in order to not get errors in the first variable. I created a copy of the database to start training the model, and after that eliminated the null values in the database.

Now we are going to create 2 new variables:

```
df['failed_login_ratio'] = df['failed_logins'] / df['login_attempts']
# Fill posible divisions by cero with 0
df['failed_login_ratio'] = df['failed_login_ratio'].fillna(0)
```

```
df['network_intensity'] = (df['bytes_sent'] + df['bytes_received']) / df['session_duration']
```

We created the variable failed_login_ratio to see an isolated number that helps us identify if the traffic is benign or an attack.

We created network_intensity so we can see the intensity of traffic in sessions, this will be useful to see what may be a probably data exfiltration session in the system.

And then we eliminate the column session_id from the dataset because it does not give useful info to out model.

```
df_clean['protocol'].fillna(df_clean['protocol'].mode(), inplace=True)
df_clean['bytes_received'].fillna(df_clean['bytes_received'].median(), inplace=True)
df_clean['signal_strength'].fillna(df_clean['signal_strength'].median(), inplace=True)
```

```
/tmp/ipykernel_23462/625133714.py:1: FutureWarning: A value is trying to be set on a copy of a DataFrame or Series through chained a
The behavior will change in pandas 3.0. This inplace method will never work because the intermediate object on which we are setting

For example, when doing 'df[col].method(value, inplace=True)', try using 'df.method({col: value}, inplace=True)' or df[col] = df[col

  df_clean['protocol'].fillna(df_clean['protocol'].mode(), inplace=True)
/tmp/ipykernel_23462/625133714.py:2: FutureWarning: A value is trying to be set on a copy of a DataFrame or Series through chained a
The behavior will change in pandas 3.0. This inplace method will never work because the intermediate object on which we are setting

For example, when doing 'df[col].method(value, inplace=True)', try using 'df.method({col: value}, inplace=True)' or df[col] = df[col

  df_clean['bytes_received'].fillna(df_clean['bytes_received'].median(), inplace=True)
/tmp/ipykernel_23462/625133714.py:3: FutureWarning: A value is trying to be set on a copy of a DataFrame or Series through chained a
The behavior will change in pandas 3.0. This inplace method will never work because the intermediate object on which we are setting

For example, when doing 'df[col].method(value, inplace=True)', try using 'df.method({col: value}, inplace=True)' or df[col] = df[col

  df_clean['signal_strength'].fillna(df_clean['signal_strength'].median(), inplace=True)
```

Activar Windows

This makes the system instead of eliminating Nan values it fills the ‘protocol’ column with the mode value (because it is

a string value (object)) and it uses the median with the numeric values such as: signal_strength and bytes_received.

```
le_clean = LabelEncoder()
cat_cols_clean = df_clean.select_dtypes(include='object').columns
for col in cat_cols_clean:
    df_clean[col] = le_clean.fit_transform(df_clean[col])
```

```
X_clean = df_clean.drop(['is_attack', 'session_duration'], axis=1)
y_clean = df_clean['is_attack']

X_train_c, X_test_c, y_train_c, y_test_c = train_test_split(
    X_clean, y_clean, test_size=0.2, random_state=42, stratify=y_clean,
)

scaler = StandardScaler()
X_train_c_sc = scaler.fit_transform(X_train_c)
X_test_c_sc  = scaler.transform(X_test_c)
```

Here we transform the text columns into number columns to make them understandable for the model.

Then we separate the model into X_clean and y_clean and y_clean is the target variable X_clean contains all other variables.

Then we divide again the decision tree model into 80/20 and train the model. Then we use standartscaler to make the numbers be between 1 and –1 and the mean will be 0, this is made to make the model treat all values with the same importance.

```
dt_better = DecisionTreeClassifier(max_depth=5, random_state=42)
dt_better.fit(X_train_c_sc, y_train_c)
y_pred_impr = dt_better.predict(X_test_c_sc)

print("Improved Decision Tree")
print(classification_report(y_test_c, y_pred_impr, target_names=['Benign', 'Attack']))
```

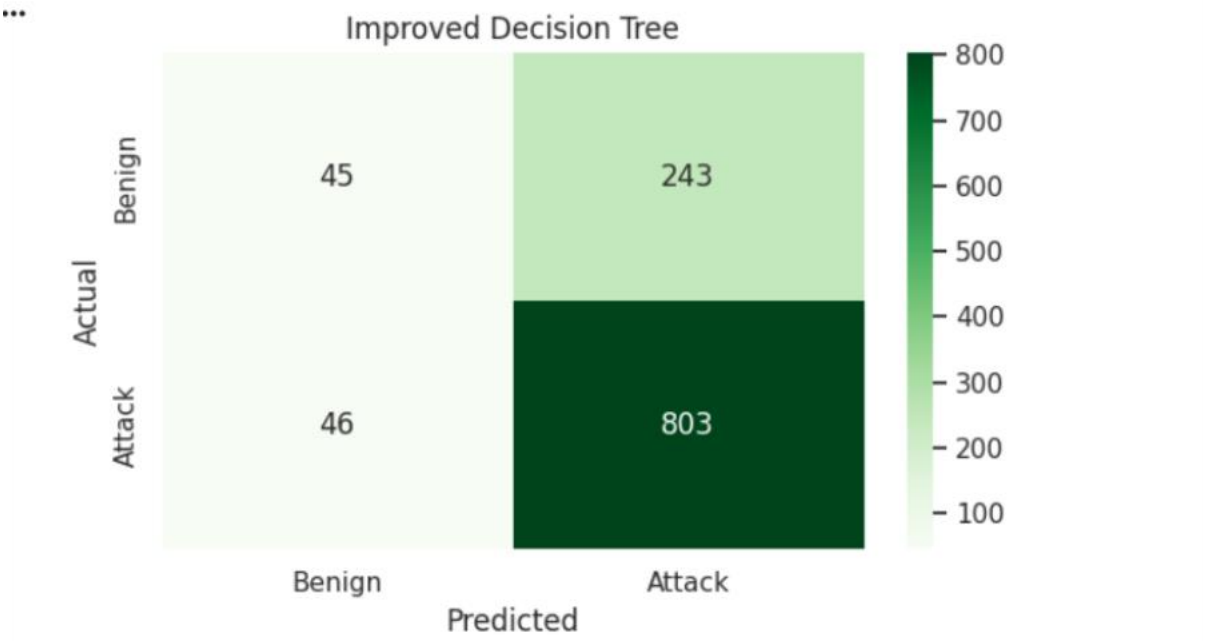
```
Improved Decision Tree
              precision    recall  f1-score   support

      Benign       0.49      0.16      0.24       288
      Attack       0.77      0.95      0.85       849

    accuracy                           0.75      1137
   macro avg       0.63      0.55      0.54      1137
weighted avg       0.70      0.75      0.69      1137
```

As we can see the model has improved, the benign precision is better being now at 49% and attack precision 77%, recall has improved for both variables as well but the model still does not do a good work when finding identifying the benign values.

This model still has a big difference between the values given for benign and for attack variables, so the model treats almost all of the benign traffic as attack traffic.



Here we can see again the number of data that the model identified correctly and the number of data that did not identify correctly, first the attack was successfully identified and the benign traffic was not, It has a 45-243 identified for benign being 45 the one that it really identified and 243 the one that was not identified.

Decision Tree comparation:

Comparing both models, it improved in the 4 different metrics used:

Precision, Recall, F1-Score and Accuracy. After doing all that data preparation the model is better, it improved and now it finds real patterns in numbers instead of just try guessing.

Improving Precision and recall simoultaneously means that the decision making has improved as well. We can see that the class benign has improved, this means that the model stopped ignoring the values that had less examples and now it tries to guess evenly.

This happened because all data was in the same scale instead of having a giant difference of values.

Prediction Model

```
X_reg = df_clean.drop(['is_attack', 'session_duration'], axis=1)
y_reg = df_clean['session_duration']
```

```
X_train_r, X_test_r, y_train_r, y_test_r = train_test_split(
    X_reg, y_reg, test_size=0.2, random_state=42
)

scaler_r = StandardScaler()
X_train_r_sc = scaler_r.fit_transform(X_train_r)
X_test_r_sc  = scaler_r.transform(X_test_r)

lr_model = LinearRegression()
lr_model.fit(X_train_r_sc, y_train_r)
y_pred_lr = lr_model.predict(X_test_r_sc)
```

Now we proceed to the prediction model:

First we eliminate the columns that do not predict or that have the same answer, then we state that we want to predict session_duration.

After this we split the database into train and test 80/20 again. Then we use StandartScaler again to make the data be between 0 and 1 because we do not want it to have very high value differences. Then we start with trainment and prediction.

```
rmse = np.sqrt(mean_squared_error(y_test_r, y_pred_lr))
r2   = r2_score(y_test_r, y_pred_lr)

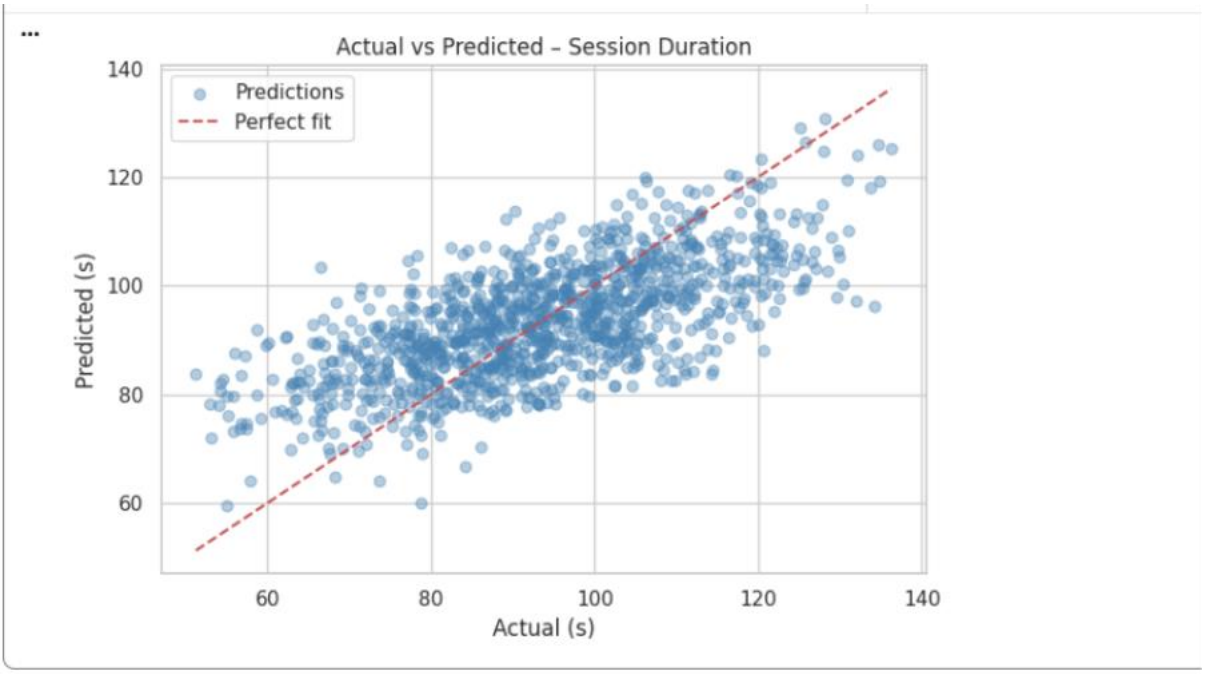
print("Prediction Model")
print(f'RMSE : {rmse:.2f}')
print(f'R²   : {r2:.4f}')
```

```
Prediction Model
RMSE : 12.05
R²   : 0.4509
```

After this we wanted to know how good the model is so we tried the regression model.

We wanted to know the RMSE (Root Mean Squared Error) which says how far are the predictions from the real values, in this case we have 12.05, this means that the model's predictions fail on average for 12.05 seconds.

Then we want to know R^2 which says the percentage of the variation that the model can explain. The model explains aproximately a 45% of it, which is not a bad result.



As we can see in the representation of the prediction model, the red line states how the perfect prediction would be. And the blue dots represent how the model's predictions really are. We can see that the model is good predicting sessions that last between 80 and 100 seconds. And where dots are above the line, the model is predicting that the session duration is more than what it really is and where it is below the line is because the session duration predicted is lower than how it really is.

https://colab.research.google.com/drive/1H3lqcmSdEYU6wM-xg0TWU4MlVzmWdWur?usp=sharing